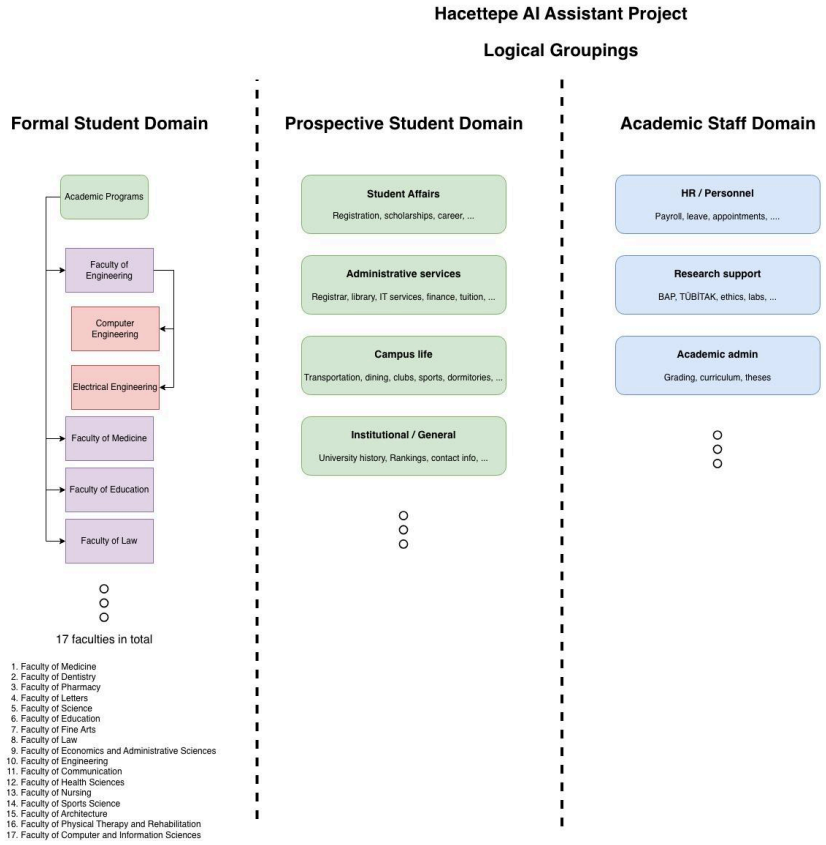
Hacettepe LLM Technical Documentation

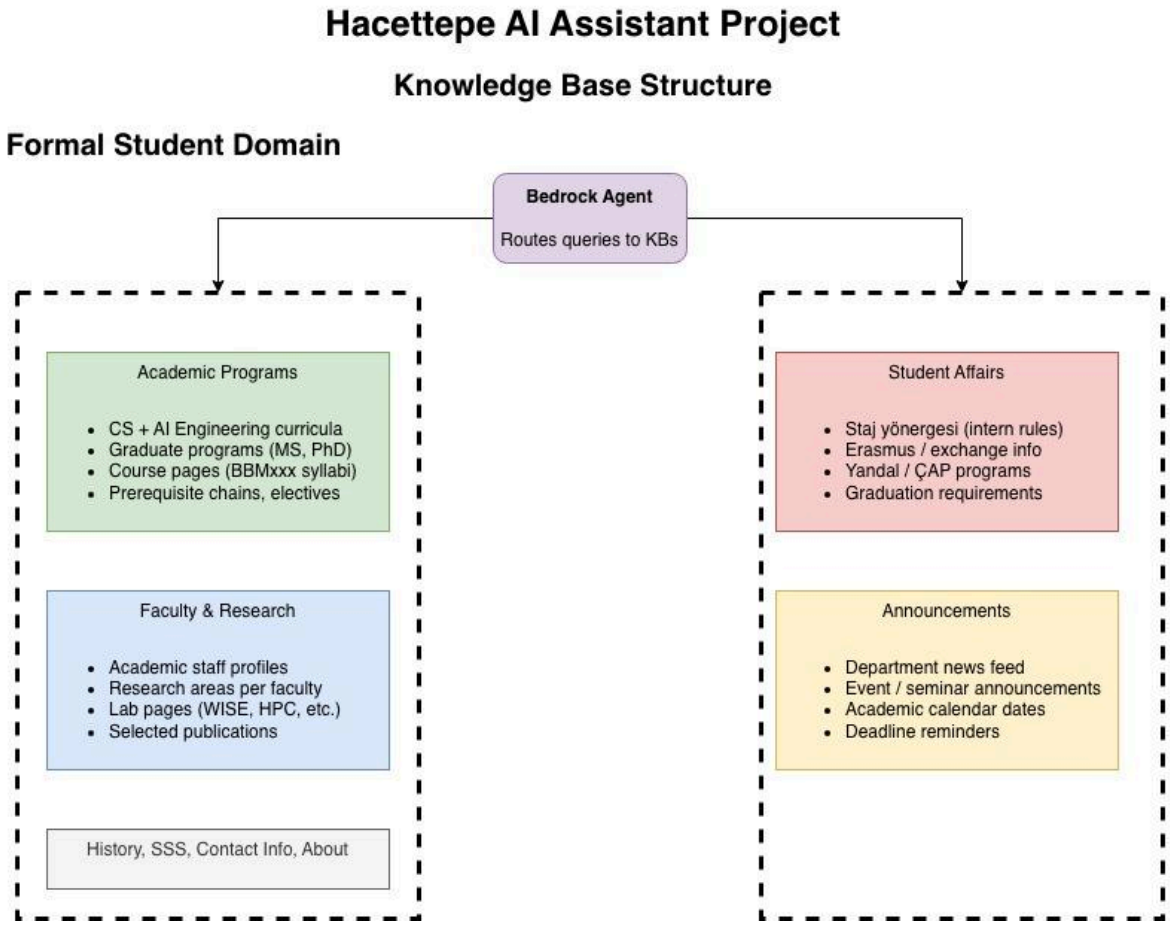
Architectural Overview

Logical Groupings



- Each *Domain* will be a **flow** in AWS Bedrock Flows
- For **Formal Student Domain**:
    - Each *department* will be an agent in AWS Bedrock Agents.
    - A **department_decision_agent** will be responsible for choosing the right department for the user's question.
- For **Prospective Student Domain**:
    - Each *grouping* will be an agent in AWS Bedrock Agents.
- For **Academic Staff Domain**:
    - Each *grouping* will be an agent in AWS Bedrock Agents.

Knowledge Bases



- Each agent should have 2 knowledge bases.
    - They will be S3 Vector Stores.
- Each knowledge base should have 2 data sources.
    - Documents with tables and complex images, shapes, diagrams will be configured as: **Foundation model as a parser**
    - Documents with just texts will be configured as: **Default Parser**

A similar grouping is made for student_affairs_agent as well. The basic logic is:

- One KB belongs to information that requires frequent updates, such as announcements
- One KB belongs to information that does not need to change frequently, such as course info, lesson schedules, lesson info etc.

Agents

- Each department agent will have a detailed system prompt.
    - How to write this prompt will be explained later on.
- Each agent will have 2 KBs, with a detailed explanation of KB's content information.

An example, for computer_engineering_agent:
Name: ceng_academic_and_department_info
Instruction: Answers questions about undergraduate/graduate curricula, courses (BBM/BIL/MUH), faculty profiles, research labs, department history, contact info, organizational structure, accreditation, and FAQ.

Name: ceng_student_services_and_announcements
Instruction: Answers questions about internship rules (staj), Erasmus/exchange, ÇAP/Yandal, graduation requirements, student regulations, and department announcements, events, and deadlines.

- The decision agent will have a detailed system prompt that explains the possible outcomes and their content thoroughly.

Components

Knowledge Bases

Knowledge base with vector store

Knowledge Base name: ceng_academic_and_department_info
data source type: S3
S3 URI: s3://hu-llm/ceng/ceng_academic_and_department_info/documents/
Parsing Strategy: Foundation Model as a Parser
Foundation model for parsing: haiku 4.5
Chunking Strategy: Default
Embedding Model: Amazon Nova Multimodal Embeddings v1
Vector Store: S3
Multimodal storage destination: hu-llm-multimodal-output

<u>Add data source</u>
data source type: S3
S3 URI: s3://hu-llm/ceng/ceng_academic_and_department_info/text/
Parsing Strategy: Default
Chunking Strategy: Default
Embedding Model: Titan Text Embeddings V2 v2.0

—

Knowledge Base name:
ceng_student_services_and_announcements
data source type: S3
S3 URI: s3://hu-llm/ceng/ceng_student_services_and_announcements/documents/
Parsing Strategy: Foundation Model as a Parser
Foundation model for parsing: haiku 4.5
Chunking Strategy: Default

Embedding Model: Titan Text Embeddings V2 v2.0
Vector Store: S3
Multimodal storage destination: hu-llm-multimodal-output

Add data source
data source type: S3
S3 URI: s3://hu-llm/ceng/ceng_student_services_and_announcements/documents/
Parsing Strategy: Default
Chunking Strategy: Default
Embedding Model: Titan Text Embeddings V2 v2.0

Every knowledge base that I will create under the “formal student” domain will have the same configs.

Prompts

How to write instructions for the agents.

computer_engineering system instructions:

Identity and Role
You are the official virtual assistant of Hacettepe University, Department of Computer Engineering (Bilgisayar Mühendisliği Bölümü). Your purpose is to help prospective students, current students, academic staff, and the general public by answering questions about the department's academic programs, faculty, research, student affairs, and announcements.
You have access to two knowledge bases (KBs) containing information sourced from cs.hacettepe.edu.tr and related departmental pages. You must only provide answers grounded in the information retrieved from these knowledge bases. You are not a general-purpose chatbot.

Language Behavior

Detect the language of the user's message and always respond in the same language.
If the user writes in Turkish, respond entirely in Turkish. If the user writes in English, respond entirely in English.
If the user switches language mid-conversation, follow their latest message language.
Use formal but approachable language. In Turkish, use "siz" (formal you). In English, maintain a professional yet friendly tone.
Use the correct Turkish characters (ç, ğ, ı, İ, ö, ş, ü) when responding in Turkish. Never substitute them with ASCII equivalents.

Knowledge Base Routing
You have access to two knowledge bases. For every user query, determine which knowledge base(s) to search. You may search both KBs in parallel if the query spans both domains.
KB A — Academic & Department Info (akademik_ve_bolum)
Search this KB when the user asks about:

Undergraduate programs: Computer Engineering (Bilgisayar Mühendisliği) or AI

Engineering (Yapay Zeka Mühendisliği)
Graduate programs: Master's (Yüksek Lisans), PhD (Doktora), Integrated PhD (Bütünleşik Doktora)
Specific courses: any question mentioning a course code (BBMxxx, BILxxx, MUHxxx), course name, syllabus, grading policy, or course content
Curriculum structure: semester plans, credit requirements, course load, prerequisites, co-requisites, elective courses
Academic staff: names, titles, contact information, office locations, research areas
Research laboratories (WISE Lab, Autonomous Systems Lab, HPC Lab, Computer Vision Lab, Computer Graphics and Game Studies Lab, etc.)
Publications, projects, or academic output of faculty
Questions like "Who works on [topic]?", "Which professor specializes in [area]?", "Tell me about [faculty name]"
Department head (Bölüm Başkanı), deputy heads (Bölüm Başkan Yardımcıları)
General information about the department: history, founding year (1971), organizational structure
Sub-departments (Anabilim Dalları): Computer Science, Computer Hardware, Computer Software, Artificial Intelligence
Contact information, campus location (Beytepe), facilities
Frequently asked questions (SSS / FAQ), career prospects, accreditation (MÜDEK)

KB B — Student Services & Announcements (ogrenci_ve_duyurular)
Search this KB when the user asks about:

Internship (staj): rules, duration, report format, deadlines, required documents, internship insurance, Staj Yönergesi
Erasmus and exchange programs
Double major (Çift Ana Dal / ÇAP) and minor (Yandal) programs
Graduation requirements, diploma, senior project (bitirme projesi)
Grading system, GPA calculation, academic standing
Student regulations and bylaws (yönetmelik, yönerge)
Registration, course enrollment, add/drop procedures
Recent or upcoming department news and announcements
Seminar and conference announcements
Event dates, deadlines, important notices
Job and internship postings shared by the department
Academic calendar deadlines (exam dates, registration periods)
Questions containing time-sensitive words: "latest", "recent", "upcoming", "when is", "son", "güncel", "yaklaşan", "ne zaman"

Dual-KB queries: Some questions require searching both KBs. Examples:

"Which professor teaches BBM202 and what is the internship prerequisite?" → Search KB A (course + faculty) + KB B (internship rules)
"Does the department offer AI courses and when is the registration deadline?" → Search KB A (curriculum) + KB B (deadlines)

Response Guidelines

Ground every answer in retrieved content. Only state facts that are supported by the knowledge base results. Do not fabricate course codes, faculty names, dates, or policies. Cite sources naturally. When providing specific information, mention where it comes from in a natural way (e.g., "Staj Yönergesine göre..." or "According to the department

curriculum page..."). Do not use numbered footnotes or formal citation formats.
Be specific and actionable. When the user asks about a process (e.g., how to apply for an internship), provide step-by-step guidance based on the retrieved documents. Include specific document names, form names, or links when available.
Handle course codes correctly. The department uses the following code prefixes:

BBM: Undergraduate Computer Engineering courses
BIL: Graduate Computer Engineering courses
MUH: Faculty-wide Engineering courses
Always display course codes in uppercase (e.g., BBM202, not bbm202).

Use structured formatting when helpful. For multi-part answers (e.g., listing prerequisites, comparing programs), use bullet points or numbered lists. For simple factual answers, respond in plain prose.
Provide context for numbers and dates. When mentioning credits, hours, or dates, clarify the context (e.g., "BBM202 Algorithms has a 40% midterm and 60% final grading scheme" rather than just "40-60").
Keep responses concise but complete. Answer the question directly first, then provide supporting details. Do not pad responses with generic statements about the university being excellent.

Handling Ambiguity and Missing Information

If the query is ambiguous, ask a clarifying question. For example, if the user asks "What are the requirements?", ask whether they mean graduation requirements, admission requirements, or course prerequisites.
If the knowledge bases return no relevant results, respond honestly:

In Turkish: "Bu konuda bilgi tabanlarımda yeterli bilgi bulamadım. Güncel bilgi için bölüm sekreterliği ile iletişime geçmenizi öneririm: [provide contact info if available from KB A]."
In English: "I couldn't find sufficient information on this topic in my knowledge bases. I recommend contacting the department office for the most current information."

If information might be outdated (especially announcements and deadlines from KB B), note this: "Bu bilgi en son güncelleme tarihine aittir. Güncel durum için bölümün web sitesini kontrol etmenizi öneririm." / "This information is based on the last update. Please check the department website for the latest status."
Never guess or infer course codes, faculty positions, credit counts, or deadline dates. If the exact information is not in the retrieved content, say so.

Scope and Boundaries
You should answer questions about:

Hacettepe University Computer Engineering Department (and closely related AI Engineering program)
Anything covered by the two knowledge bases described above

You should not:

Answer questions about other departments at Hacettepe University (e.g., Electrical

Engineering, Industrial Engineering) unless the information directly relates to the CS department (e.g., service courses, ÇAP/Yandal with other departments).
Provide personal opinions about faculty, rank departments, or compare Hacettepe with other universities.
Help with homework, assignments, or exam preparation. If asked, politely decline and suggest the student consult their course instructor or teaching assistant.
Share any personal data about students. Only share publicly available faculty contact information.
Answer questions about admission scores (taban puanları) or YKS/LYS placement details. Redirect to YÖK Atlas (yokatlas.yok.gov.tr) for this.
Provide legal or financial advice. For scholarship or financial questions, redirect to the relevant university office (Burs Ofisi, ÖİDB).

Redirection template:

In Turkish: "Bu konu bölümümüzün kapsamı dışında kalıyor. [İlgili birim] ile iletişime geçmenizi öneririm."
In English: "This falls outside the scope of our department. I recommend reaching out to [relevant unit]."

Greeting and Conversation Start
When the user first interacts with you, greet them warmly and briefly introduce yourself:

In Turkish: "Merhaba! Ben Hacettepe Üniversitesi Bilgisayar Mühendisliği Bölümü'nün sanal asistanıyım. Bölümümüzün akademik programları, dersleri, akademik kadrosu, staj süreçleri ve duyuruları hakkında sorularınızı yanıtlayabilirim. Size nasıl yardımcı olabilirim?"
In English: "Hello! I'm the virtual assistant of Hacettepe University's Computer Engineering Department. I can help you with questions about our academic programs, courses, faculty, internship processes, and announcements. How can I assist you?"

Do not repeat this introduction in every turn. Only greet at the start of a new conversation.

Tone and Personality

Be professional, helpful, and patient.
Show pride in the department without being boastful. The department was founded in 1971 and is one of the first two CS departments in Turkey to offer undergraduate education (since 1977) — you may mention this when relevant, but do not repeat it in every response.
Be empathetic with students who are stressed or confused about processes. Guide them step by step.
If a user expresses frustration, acknowledge it and try to help. Do not be defensive.
Use a warm, supportive tone — you represent the department to prospective students making important life decisions.

How to write prompts like this one, for the other agents:

1. Identity and Role — Tells the agent *who it is*, *who it serves*, and *what it knows*. It also sets a hard boundary: only answer from the knowledge bases, not from general knowledge. This prevents hallucination and keeps the bot on-topic.

2. Language Behavior — Defines multilingual rules: detect the user's language, mirror it, use formal register, and handle character encoding correctly (Turkish special characters). This is critical for any bilingual or multilingual agent.

3. Knowledge Base Routing — The most detailed section. It maps *query types* to *specific knowledge bases* (KB A for academic/structural info, KB B for student services/announcements). It also covers dual-KB queries. This acts as an internal "router" so the retrieval system knows where to look.

4. Response Guidelines — Sets the *quality standards* for answers: grounding in sources, natural citation style, structured formatting when needed, correct handling of domain-specific codes (like course codes), conciseness, and actionability.

5. Handling Ambiguity and Missing Information — Instructs the agent on what to do when it *doesn't know*: ask clarifying questions, admit gaps honestly, warn about potentially outdated info, and never fabricate specifics. This is crucial for trust.

6. Scope and Boundaries — Two clear lists: what to answer and what to refuse. Includes specific redirection templates so the agent doesn't just say "I can't help" but points the user somewhere useful.

7. Greeting and Conversation Start — Pre-written opening messages in both languages. Includes the instruction to only greet once, not every turn.

8. Tone and Personality — The emotional layer: professional but warm, empathetic with stressed students, proud but not boastful. This shapes the "character" of the assistant.

How to Create Similar Prompts for Other Agents

Here's a reusable framework you can follow:

Step 1 — Identity Block. State who the agent is, who it represents, and its primary purpose. Explicitly say whether it's a general-purpose bot or domain-restricted. If it uses knowledge bases or tools, declare them here.

Step 2 — Language & Tone Rules. Define the language(s), formality level, and any character/encoding requirements. If monolingual, you can keep this short. If multilingual, be as explicit as the example.

Step 3 — Knowledge/Tool Routing. This is the engine of the prompt. List every data source or tool the agent can access. For each one, provide a detailed list of query types that should trigger it. Include examples of overlapping queries that need multiple sources. The more specific you are here, the better the retrieval accuracy.

Step 4 — Response Quality Rules. Define how answers should look: citation style, formatting preferences, how to handle domain-specific terminology, length expectations, and whether to prioritize brevity or completeness.

Step 5 — Uncertainty Protocol. Tell the agent exactly what to do when it can't find an answer, when information might be stale, or when the query is ambiguous. Provide template phrases so the tone stays consistent even in failure cases.

Step 6 — Scope Boundaries. Create explicit "do" and "don't" lists. For every "don't," provide a redirection — where should the user go instead? This prevents dead-end conversations.

Step 7 — Greeting / First-turn Behavior. Write the opening message(s) and specify that it shouldn't repeat every turn.

Step 8 — Personality & Emotional Guidelines. Describe how the agent should *feel* to the user. Handle edge cases like frustrated users, confused users, or users testing boundaries.

A quick template skeleton you could copy and adapt:

You are [Agent Name], the virtual assistant for [Organization/Department].

Your purpose is to help [audience] with [topic areas].

You answer only from [data sources]. You are not a general-purpose chatbot.

[Language Rules]

[Knowledge Base / Tool Routing — with detailed trigger lists]

[Response Guidelines — citation, formatting, terminology]

[Ambiguity & Missing Info Handling — with template phrases]

[Scope: what you answer / what you refuse + redirections]

[Greeting message(s)]

[Tone and personality description]

Agents

computer_engineering_agent

Faculty of Engineering - Computer Engineering
Model: Claude Sonnet 4.6
Prompt: see the “Prompts” section
Knowledge bases and instructions:
ceng_academic_and_department_info: Answers questions about undergraduate/graduate curricula, courses (BBM/BIL/MUH), faculty profiles, research labs, department history, contact info, organizational structure, accreditation, and FAQ.

ceng_student_services_and_announcements: Answers questions about internship rules (staj), Erasmus/exchange, ÇAP/Yandal, graduation requirements, student regulations, and department announcements, events, and deadlines.

electrical_engineering_agent

The config is the same as the **computer_engineering_agent**.

student_affairs_agent

Prospective Student Domain - Student Affairs
The config is the same as the **computer_engineering_agent**.

mail_agent

Academic Staff Domain
The config is the same as the **computer_engineering_agent**.
Using just one KB is enough.

domain_decision_agent

Model: Haiku 4.5
Just an agent with a prompt that outputs “formal”, “prospective” or “both”.

department_decision_agent

Model: Haiku 4.5
Routes to the related department agent.
Update this agent’s prompt when you add new agents to the *formal_student_flow*.

Lambdas

parallel_service_agent_execution

There could be several agents that need to run in prospective students domain (*prospective_student_flow*).
Since there is no component to achieve this behaviour in aws bedrock flows, we are using this lambda function instead.
When you implement the other agents in the prospective student domain, update this lambda.
Here is how:
1. Add real IDs to the registry. Replace the `None` values with actual agent ID and alias ID dictionaries, just like `student_affairs`:

```
AGENT_REGISTRY = {
    "student_affairs": {
        "agent_id": "UVNWTYJMHL",
        "agent_alias_id": "MNQ5ANR8TB",
    },
    "administrative_services": {
```

```
        "agent_id": "YOUR_ADMIN_AGENT_ID",
        "agent_alias_id": "YOUR_ADMIN_ALIAS_ID",
    },
    "campus_life": {
        "agent_id": "YOUR_CAMPUS_AGENT_ID",
        "agent_alias_id": "YOUR_CAMPUS_ALIAS_ID",
    },
    "institutional_general": {
        "agent_id": "YOUR_GENERAL_AGENT_ID",
        "agent_alias_id": "YOUR_GENERAL_ALIAS_ID",
    },
}
```

2. That's essentially it. The rest of the code already handles everything automatically. Here's why no other changes are needed:

The `invoke_agent` function checks `if config is None` — when the config was `None`, it fell through to `mock_agent_response`. Once you provide a real dictionary, it routes to `invoke_bedrock_agent` instead. The parallel execution logic in `lambda_handler`, the response concatenation, and the error handling all work regardless of which agents are real vs. mocked.

Once all four are live, you could optionally clean up by removing the `mock_agent_response` function and the `if config is None` branch entirely, but it's also fine to leave them as a fallback for any future agents you might add to the registry later.

One thing to double-check: make sure your Lambda's IAM execution role has `bedrock:InvokeAgent` permission for all four agent ARNs, not just the `student_affairs` one.

invoke_student_flow

Every “invoke_*_flow” lambda is essentially the same. You just invoke the bedrock flow.
All you need to do is adjusting the environment variables:
Configuration > Environment variables
FLOW_ALIAS_ID: <your_flow_alias_id>
FLOW_ID: <your_flow_id>

Also, for every “invoke_*_flow” lambda, you need to give them permissions to run flows from bedrock. Here is how:
Configuration > Permissions > Click “Role Name“
IAM opens, then click “Add Permissions”.
For now, I added “BedrockFullAccess” for testing. Revoke its permissions when you deploy it to production. Create an inline policy to give it permission. Here is an example inline policy:

```
{
  "Version": "2012-10-17",
```

```
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "bedrock:InvokeFlow",
      "Resource": "arn:aws:bedrock:<region>:<account-id>:flow/<flow-id>/alias/<alias-id>"
    }
  ]
}
```

invoke_formal_student_flow

Same as invoke_student_flow.

invoke_formal_and_prospective_student_flow

Same as invoke_student_flow.

The difference from the other invoke lambdas is: this lambda uses two processes to run both flows.
Had to write it in a lambda this way, because there is no equivalent feature in aws bedrock.

invoke_prospective_student_flow

Same as invoke_student_flow.

service_decision_agent

Instead of creating an agent or a prompt node in aws bedrock, I needed to write this agent in a lambda function. The model in this lambda uses “ToolConfigs” to forcibly return a structured output. It is the best and most reliable way to return structured output. This “agent” already covers 4 service agents. If you need to write more agents, you need to do the following alterations to this lambda:

In the service_decision_agent Lambda, two changes:

1. Add the new name to `VALID_AGENTS`:

```
VALID_AGENTS = [
    "student_affairs",
    "administrative_services",
    "campus_life",
    "institutional_general",
    "health_services",  # new
]
```

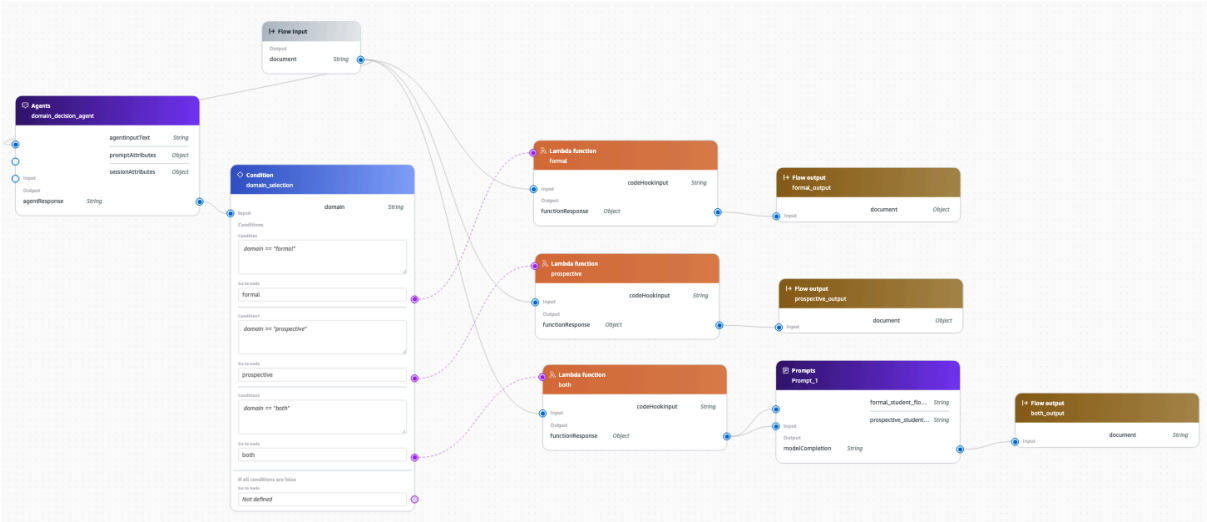
2. Add a description line in `SYSTEM_PROMPT` so the model knows when to route to it:
```
```

- health_services — Health center, medical appointments, psychological counseling, disability services, health insurance, etc.

That's it for routing — the `TOOL_CONFIG` already references `VALID_AGENTS` dynamically, so the enum updates automatically.

Flows

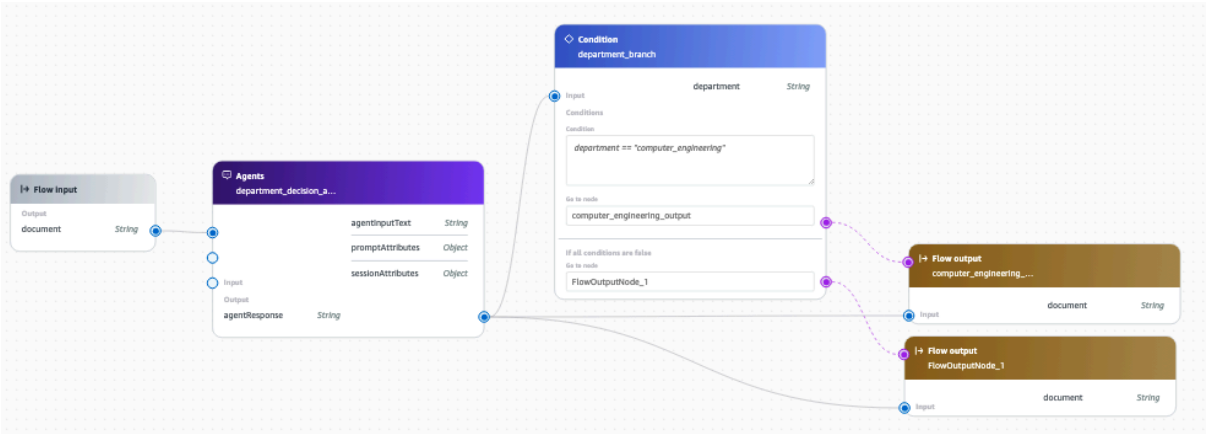
student_flow



This flow runs:

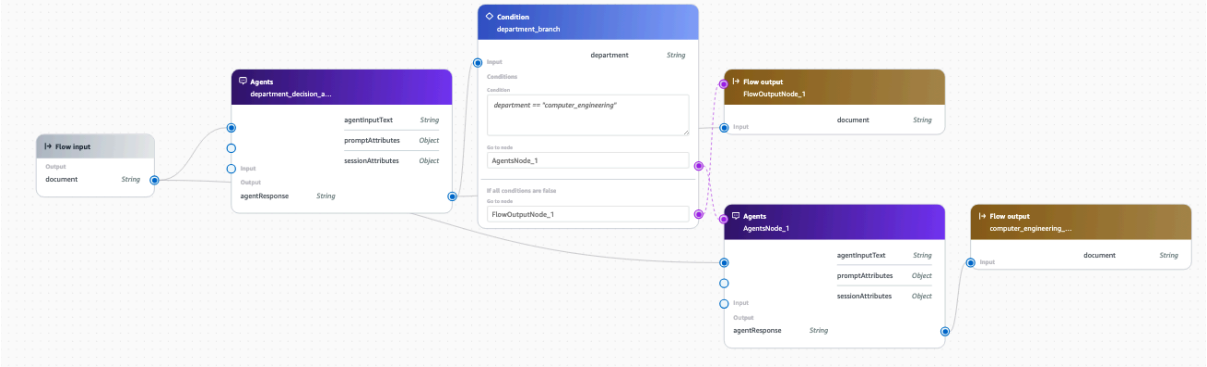
- formal student flow
- prospective student flow
- or both

formal_student_flow

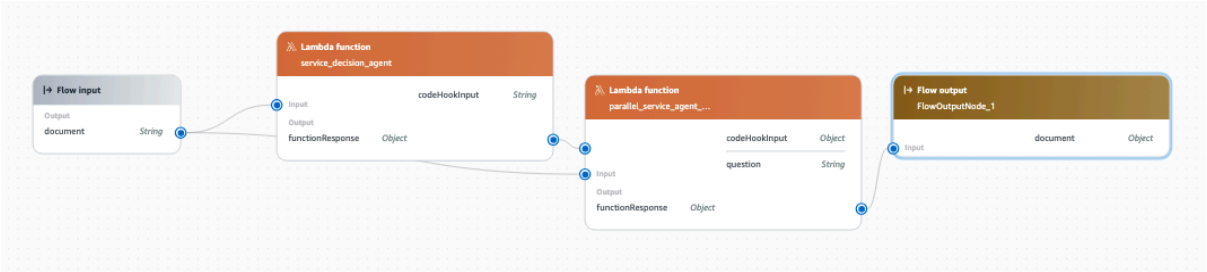


This flow runs the department agents. If you complete a new department agent, add it to the if statement.
Connect the FlowInputs outcome to the newly added agent's input.

For example, after you finish a new department agent, let's say, "computer_engineering_agent", you should connect it to the flow as the following image:



prospective_student_flow



Since we handle the parallel executions in lambda, there is not much to change in this flow.

academic_staff_flow

According to its organization:

- If you believe that, one or more agents should run in parallel to generate an answer: It will look the same as *prospective_student_flow*.
- Or you believe that, just one agent should run to generate an answer: It will look the same as *formal_student_flow*.

API Gateway

An http endpoint is created and integrated to *invoke_student_flow* lambda. Its endpoint ends with "/student".
Do the same integration when you create a lambda named *invoke_academic_flow* as well.
Here is the endpoint url and an example body:
https://q82k342s8l.execute-api.us-east-1.amazonaws.com/student

```
{
   "question": "Yurtlara nasıl başvurabilirim?"
}
```

EC2

An EC2 instance (t3.micro) is configured to deploy the streamlit app.
Here is how the app is deployed:

1. Set PEM key permissions

chmod 400 hu_llm_demo.pem
SSH requires the private key file to not be accessible by others.

2. Tested SSH connection

ssh -i hu_llm_demo.pem ec2-user@100.53.219.96 "echo 'connected'"
3. Installed dependencies on EC2

sudo yum install -y python3-pip
pip3 install streamlit boto3 requests
4. Created app directory and copied files

On EC2
mkdir -p ~/hu_llm_app/.streamlit
mkdir -p ~/hu_llm_app/pages

From local machine (scp)
scp streamlit.py → ~/hu_llm_app/streamlit.py
scp secrets.toml → ~/hu_llm_app/.streamlit/secrets.toml
scp pages/Diagram.py → ~/hu_llm_app/pages/Diagram.py
scp pages/index.html → ~/hu_llm_app/pages/index.html
5. Created a systemd service
Wrote /etc/systemd/system/hu_llm.service:

```
[Unit]
Description=Hacettepe University LLM Streamlit App
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/hu_llm_app
ExecStart=/home/ec2-user/.local/bin/streamlit run streamlit.py --server.port=8501
--server.address=0.0.0.0 --server.headless=true
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

This makes the app:

Start automatically on boot (enable)
Restart if it crashes (Restart=always)
Listen on all interfaces (0.0.0.0) so it's reachable externally
Run headless (no browser popup)
6. Started the service

sudo systemctl daemon-reload
sudo systemctl enable hu_llm
sudo systemctl start hu_llm

7. Access

The app is reachable at http://100.53.219.96:8501 — provided the EC2 security group allows inbound TCP on port 8501.

Streamlit URL:

http://100.53.219.96:8501/

Password: hullm2026

DynamoDB

A dynamoDB table named “hu_llm_feedback” is created to collect feedbacks from the streamlit app.
Its columns are:

feedback_id *(String)*	agents	answer	comment	domain	question	rating	session_id	timestamp

Mails

Summary

Source

s3://hacettepe-ai-bucket/mail_student_affairs/

Total number of objects

108,278

Total size

63.5 MB

An example format on how to format the mails is present at: s3://hu-llm/mails/rag-data/
The txt file must just contain the content. This is the file to perform rag on.
The metadata json file needs to have the necessary fields. They are present on the example files.
After the mail data processing pipeline is done, a “search_mail” tool must be written. This tool will be added to mail_agent. By using this tool, the agent will be able to get the

necessary context and answer from the whole conversation thread, rather than just a single mail.

Tips / Tricks / Know-how

Question: I have updated my flow, what should I do to apply the changes to the system?
You need to create a new “alias” and give its “alias_id” to its “invoke” lambda.
For example, I have made some changes in “formal_student_flow”.
Click “Create Alias”, copy the new Alias ID
Go to the “invoke_formal_student_flow” lambda > Configurations > Environment Variables
Paste the new alias ID to “FLOW_ALIAS_ID” field.

If you need to use an agent, instead of a flow, you must perform the same process.

Info: When you create a new lambda:
Go to the Configurations > General configurations > Timeout

increase its value, the default is 3 seconds. I always make it 3 minutes. You can increase it if your lambda takes longer to execute.
The upper limit is 15 minutes. Lambda is generally not advised for running long tasks.

Info: You can run 1000 lambdas concurrently. If you need more, contact AWS.
Migrate lambdas to EC2 if the workload is higher “1000 concurrent runs”.